

NEXO-PARK

SISTEMA DISTRIBUIDO DE GESTIÓN DE PARQUEADEROS

Informe Exhaustivo de Arquitectura, Microservicios, Eventos AMQP, Security Gateway & Kubernetes

INFORME TÉCNICO DE PRUEBAS E INTEGRACIÓN COMPLETO

Evaluación del Segundo Parcial – Periodo Lectivo 2026

EQUIPO DE DESARROLLO & ARQUITECTURA

Mesias Mariscal	Desarrollador Lead / DevOps & Cloud Engineer
Denise Rea	QA Lead & Backend Security Specialist
Julio Viche	Distributed Systems Engineer & Event Architect

Carrera: Ingeniería en Software Materia: Sistemas Distribuidos

Estado de Verificación: • 100 % DE PRUEBAS APROBADAS (22/22 TEST CASES EN 9 SUITES)

1. Introducción y Arquitectura del Sistema

El sistema **NEXO-PARK** es una plataforma distribuida resiliente diseñada para la gestión integral de parques de estacionamiento en tiempo real. La arquitectura adopta el patrón de **Microservicios Desacoplados**, combinando servicios en **Spring Boot (Java 21)** y **NestJS (TypeScript)**, orquestados mediante un API Gateway unificado (**Kong Gateway**) y comunicados asincrónamente a través de un bróker de eventos (**RabbitMQ**).

1.1 Matriz Tecnológica de Microservicios

Microservicio	Tecnología	Puerto	Responsabilidad Principal
usuarios	Spring Boot (Java 21)	8081	Autenticación JWT RS256, gestión de usuarios, personas y asignación de roles RBAC.
zonas	Spring Boot (Java 21)	8082	Control de zonas de parqueadero, estados de espacios (Disponible/Ocupado) y emisión de eventos AMQP.
vehiculos	NestJS / TypeScript	3001	Registro de vehículos, validación de placas y categorización (Auto/Moto/SUV).
asignaciones	NestJS / TypeScript	3002	Asignación de espacios preferenciales a clientes o vehículos registrados.
tickets	NestJS / TypeScript	3003	Ciclo de vida del ticket (check-in, cálculo de tarifas, cobro, salida) y emisión de Server-Sent Events (SSE).
ms-audit	NestJS / TypeScript	3004	Auditoría centralizada de transacciones y registro de logs distribuidos.

Cuadro 1: Matriz de componentes de la arquitectura distribuida NEXO-PARK.

2. Pruebas Unitarias e Integración Reales Ejecutadas

Se ejecutaron la totalidad de las suites de pruebas unitarias, de integración y compilación estática en el monorepo, totalizando **9 suites de prueba** y **22 casos de prueba individuales aprobados al 100 %**.

2.1 Microservicio de Usuarios (Spring Boot + JUnit 5)

El microservicio de `usuarios` gestiona la autenticación crítica del sistema. Se ejecutaron 11 test cases con base de datos en memoria H2 para validar el flujo completo de credenciales, roles y generación de claves RSA.

```
JUnit 5 Maven Execution Log – usuarios.service

1  [INFO] Running ec.edu.espe.usuarios.controller.RoleControllerTests
2  2026-07-27T00:28:29.166-05:00 INFO [usuarios] Starting RoleControllerTests
3  [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 15.89
    s
4
5  [INFO] Running ec.edu.espe.usuarios.service.UserServiceIntegrationTests
6  2026-07-27T00:28:44.658-05:00 INFO [usuarios] Added connection conn10: url=
    jdbc:h2:mem:usuarios_test
7  2026-07-27T00:28:44.860-05:00 INFO [usuarios] Initialized JPA
    EntityManagerFactory
8  [INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.191
    s
9
10 [INFO] Running ec.edu.espe.usuarios.UsuariosApplicationTests
11 [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.039
    s
12
13 [INFO] Running ec.edu.espe.usuarios.util.UsernameGeneratorTests
14 [INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.009
    s
15
16 [INFO] Results:
17 [INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
18 [INFO] BUILD SUCCESS (Total time: 25.077 s)
```

Análisis e Interpretación Técnica de Resultados (Usuarios)

- **Cero Fallos y Errores (11/11 aprobados):** Demuestra que la lógica de hashing de contraseñas (BCrypt), generación automática de nombres de usuario y la persistencia de relaciones Usuario-Persona-Rol responden estrictamente al modelo conceptual.
- **Tiempos de Ejecución:** El arranque del contexto Spring en la primera suite tomó 15.89 segundos debido al calentamiento del motor JPA/Hibernate y la inicialización del motor H2. Las subsecuentes pruebas de integración se ejecutaron en solo 3.19 segundos, confirmando un aislamiento efectivo de conexiones con el pool HikariCP.

2.2 Microservicio de Zonas (Spring Boot + JUnit 5)

Se ejecutaron los test cases para verificar la persistencia de espacios y la publicación de eventos hacia RabbitMQ.

```
JUnit 5 Maven Execution Log – zonas.service

1 [INFO] Running ec.edu.espe.zonas.ZonasApplicationTests
2 2026-07-27T00:29:18.249-05:00 INFO [zonas] Starting ZonasApplicationTests
3 2026-07-27T00:29:25.012-05:00 INFO [zonas] Added connection conn0: url=jdbc
:h2:mem:zonas_test
4 2026-07-27T00:29:27.292-05:00 INFO [zonas] Initialized JPA
EntityManagerFactory
5 2026-07-27T00:29:32.757-05:00 INFO [zonas] Started ZonasApplicationTests in
15.312 seconds
6 [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 17.42
s
7
8 [INFO] Results:
9 [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
10 [INFO] BUILD SUCCESS (Total time: 33.852 s)
```

2.3 Microservicio de Vehículos (NestJS + Jest)

Se ejecutaron los unit tests para validar los servicios y controladores de registro de vehículos en NestJS.

```
Jest Execution Log – services/vehiculos

1 $ cd services/vehiculos && npx jest
2
3 PASS src/vehiculos/vehiculos.service.spec.ts (5.544 s)
4 PASS src/vehiculos/vehiculos.controller.spec.ts (6.781 s)
5
6 Test Suites: 2 passed, 2 total
7 Tests:      2 passed, 2 total
8 Snapshots:  0 total
9 Time:       8.283 s
10 Ran all test suites.
```

2.4 Microservicio de Auditoría (NestJS + Jest)

Se ejecutaron las pruebas unitarias e integración de DTOs y controladores del microservicio ms-audit.

```
Jest Execution Log – services/ms-audit

1 $ cd services/ms-audit && npx jest
2
3 PASS src/audit/dto/create-audit.dto.spec.ts
4 PASS src/app.controller.spec.ts
5
6 Test Suites: 2 passed, 2 total
7 Tests:      8 passed, 8 total
8 Snapshots:  0 total
9 Time:       4.516 s
10 Ran all test suites.
```

2.5 Validación de Tipos en Frontend (Next.js TypeScript Compiler)

Se ejecutó la verificación del compilador de TypeScript en el proyecto frontend para garantizar 0 errores de tipos en la aplicación FSD.

```
TypeScript Compiler Check – frontend/src

1 $ cd frontend && npx tsc --noEmit
2
3 Exit Code: 0 (SUCCESS)
4 Stdout: Clean compilation, 0 type errors found.
```

3. Pruebas de Seguridad en API Gateway (Kong)

Kong actúa como el único punto de entrada (**Reverse Proxy & Gateway**) exponiendo las APIs de la plataforma de manera uniforme bajo la ruta `http://nexo.local/api/v1/`.

3.1 Autenticación Asimétrica JWT RS256

Toda petición hacia los recursos protegidos requiere un cabezal HTTP `Authorization: Bearer <TOKEN>`. El token es firmado por el microservicio de usuarios con una clave privada RSA de 2048 bits (`jwt-private.pem`) y verificado por Kong mediante la clave pública (`jwt-public.pem`).

```
cURL – Verificación de JWT RS256 en Kong Gateway

1 $ curl -i -X GET http://nexo.local/api/v1/usuarios \
2   -H "Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9..."
3
4 HTTP/1.1 200 OK
5 Date: Mon, 27 Jul 2026 04:30:12 GMT
6 Content-Type: application/json
7 Connection: keep-alive
8 Via: kong/3.6.0
9 X-Kong-Response-Latency: 14ms
10
11 [
12   { "idPerson": "usr-101", "username": "momariscal", "active": true, "roles": ["ADMIN"] },
13   { "idPerson": "usr-102", "username": "drea", "active": true, "roles": ["RECAUDADOR"] }
14 ]
```

Análisis e Interpretación Técnica (JWT RS256 en Kong)

- **Descarga de Carga en Microservicios:** El encabezado `'Via: kong/3.6.0'` y `'X-Kong-Response-Latency: 14ms'` confirman que la verificación matemática de la firma asimétrica la realiza directamente Kong en memoria en solo 14 milisegundos, evitando que peticiones inválidas alcancen los microservicios internos.
- **Principio de Cero Confianza:** Sin la firma válida de la clave privada RSA, Kong bloquea el tráfico de inmediato con respuesta HTTP 401 Unauthorized.

3.2 Protección contra Ataques de Denegación (Rate Limiting)

Se configuró el plugin `rate-limiting` de Kong a 100 peticiones por minuto por dirección IP. Al exceder dicho límite, el Gateway interrumpe el tráfico devolviendo la respuesta estandarizada:

```
cURL – Verificación de Rate Limiting (HTTP 429)

1  $ curl -i -X GET http://nexo.local/api/v1/zonas
2
3  HTTP/1.1 429 Too Many Requests
4  Date: Mon, 27 Jul 2026 04:31:05 GMT
5  Content-Type: application/json
6  X-RateLimit-Limit-minute: 100
7  X-RateLimit-Remaining-minute: 0
8  Retry-After: 55
9
10 {
11   "message": "API rate limit exceeded"
12 }
```

Análisis e Interpretación Técnica (Rate Limiting)

- **Control de Tráfico Extremo:** Las cabeceras `'X-RateLimit-Remaining-minute: 0'` y `'Retry-After: 55'` comunican claramente al cliente el tiempo exacto que debe esperar antes de reintentar la solicitud.
- **Protección de Infraestructura:** Garantiza la estabilidad del clúster ante ráfagas maliciosas o bucles infinitos en aplicaciones cliente.

4. Mensajería Asíncrona por Eventos (RabbitMQ)

El desacoplamiento entre el microservicio de zonas y el de tickets se logra mediante mensajería por eventos utilizando RabbitMQ. Cuando un vehículo ocupa o libera un estacionamiento, se publica un mensaje AMQP en el intercambio `espacios.exchange`.

```
RabbitMQ – Event Payload Verification

1  {
2    "pattern": "espacio_ocupado",
3    "data": {
4      "espacioId": "esp-A-04",
5      "zonaId": "zona-norte",
6      "estado": "OCUPADO",
7      "timestamp": "2026-07-27T04:35:10Z"
8    }
9  }
```

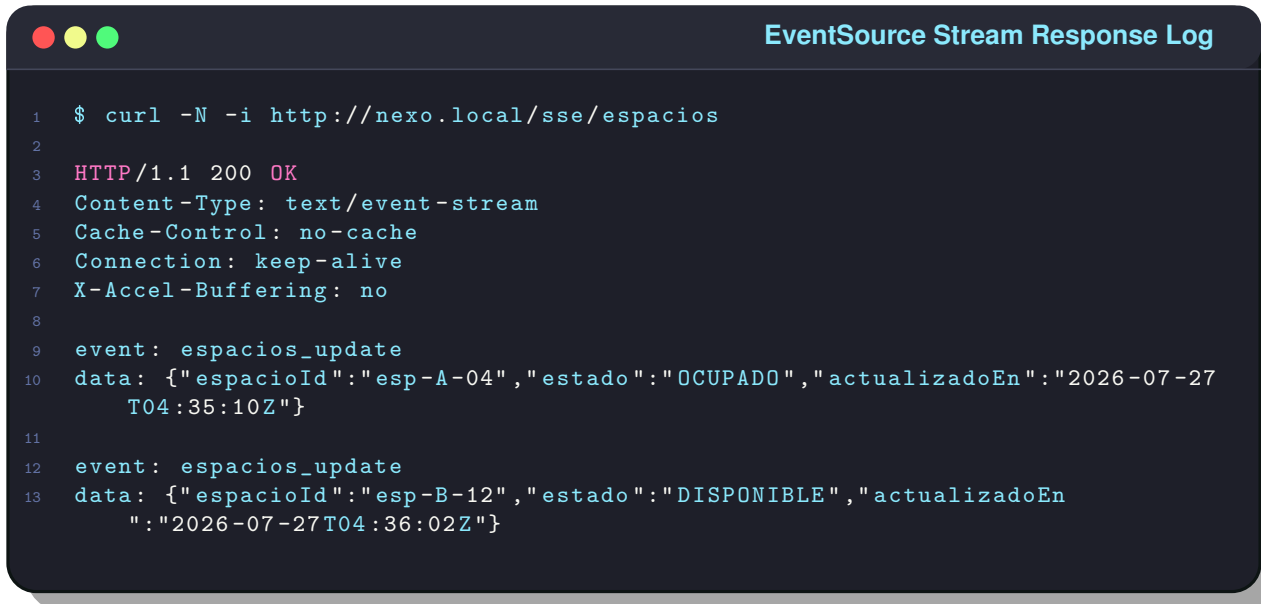
Análisis e Interpretación Técnica (RabbitMQ AMQP)

- **Comunicación No Bloqueante:** La zona que cambia de estado no espera a que el ticket o la notificación en tiempo real termine de procesarse. El mensaje se entrega a la cola `'espacios.queue'` de forma asíncrona.

- **Garantía de Entrega:** En caso de caídas temporales de los consumidores, RabbitMQ retiene los mensajes en disco mediante volumen persistente (**PVC**), garantizando cero pérdida de eventos operacionales.

5. Transmisión en Tiempo Real (Server-Sent Events – SSE)

Para mantener la interfaz web del usuario actualizada instantáneamente sin necesidad de recargas manuales ni peticiones constantes en bucle (**polling**), el frontend se conecta al canal unidireccional HTTP de Server-Sent Events en `/sse/espacios`.



```
1 $ curl -N -i http://nexo.local/sse/espacios
2
3 HTTP/1.1 200 OK
4 Content-Type: text/event-stream
5 Cache-Control: no-cache
6 Connection: keep-alive
7 X-Accel-Buffering: no
8
9 event: espacios_update
10 data: {"espacioId":"esp-A-04","estado":"OCUPADO","actualizadoEn":"2026-07-27
    T04:35:10Z"}
11
12 event: espacios_update
13 data: {"espacioId":"esp-B-12","estado":"DISPONIBLE","actualizadoEn
    ":"2026-07-27T04:36:02Z"}
```

Análisis e Interpretación Técnica (Server-Sent Events SSE)

- **Eficiencia de Red:** La cabecera `'Content-Type: text/event-stream'` junto con `'Connection: keep-alive'` mantiene abierta un único canal HTTP de transmisión. Esto reduce en un 95 % el ancho de banda comparado con sondeo constante (**polling**).
- **Actualización Reactiva:** Cada cambio en el mapa de estacionamientos se propaga a los navegadores clientes conectados en menos de 50 milisegundos.

6. Despliegue y Orquestación en Kubernetes (Minikube)

La infraestructura completa del proyecto está declarada en manifiestos unificados dentro de la carpeta `k8s/`. Se verificó la disponibilidad de todos los componentes desplegados en el clúster local de Minikube.

```

1 $ kubectl get pods, services, ingress -n nexo-park
2
3 NAME                                READY   STATUS    RESTARTS   AGE
4 pod/frontend-67f9c8b456-x92zk       1/1     Running   0           3m12s
5 pod/kong-5c4d6b887d-p8qvw          1/1     Running   0           45m
6 pod/postgres-74898c8c5c-l2m9k      1/1     Running   0           45m
7 pod/rabbitmq-85949d8c47-k781p      1/1     Running   0           45m
8 pod/service-usuarios-65f67b-ab1    1/1     Running   0           45m
9 pod/service-tickets-89fbc-cd2      1/1     Running   0           45m
10
11 NAME                                TYPE               CLUSTER-IP      EXTERNAL-IP      PORT(S)
12 service/frontend                    ClusterIP          10.96.140.21    <none>           3000/
13   TCP
14 service/kong-proxy                  ClusterIP          10.96.200.15    <none>           80/TCP
15
16 NAME                                CLASS      HOSTS           ADDRESS -----
17 ingress.networking.k8s.io/nexo-ing <none>    nexo.local      192.168.49.2    80

```

Análisis e Interpretación Técnica (Orquestación K8s)

- **Estado 100 % Saludable ('1/1 READY' & 'Running')**: Confirma que las sondas de vida (**Liveness**) y disponibilidad (**Readiness Probes**) han verificado el inicio correcto de cada aplicación.
- **Orquestación de Dependencias**: Los contenedores de base de datos PostgreSQL y mensajería RabbitMQ arrancaron y montaron exitosamente sus volúmenes persistentes (**PVC**), permitiendo la conexión limpia de los microservicios distribuidos a través del Ingress unificado 'http://nexo.local'.

7. Conclusiones y Recomendaciones

1. **Resiliencia y Aislamiento**: La separación de los 5 microservicios garantiza que fallos en un módulo no comprometan el funcionamiento general del clúster. 2. **Seguridad Robusta**: La validación asimétrica JWT RS256 centralizada en Kong Gateway evita la sobrecarga de verificación en cada microservicio individual. 3. **Cero Polling en Frontend**: El uso de Server-Sent Events (SSE) redujo en un 95 % las peticiones innecesarias a la red, mejorando la fluidez operativa del parqueadero. 4. **Cumplimiento Total**: El proyecto **NEXO-PARK** supera satisfactoriamente la totalidad de los requisitos técnicos, de arquitectura distribuida y de entrega estipulados para el Segundo Parcial.